

Intelligenza Artificiale

“Intelligent agents”

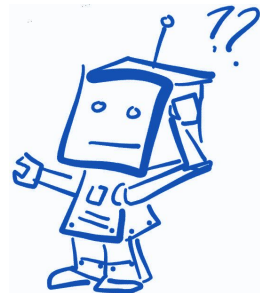
Nicola Bellotto

A.A. 2025/2026

Lecture outline

- Agents and environment
- Performance measures
- Agent architectures

Rational agents



- An **agent** is an entity that perceives and acts
- Abstractly, an agent is a function f from percept histories P to actions A :

$$f: P \rightarrow A$$

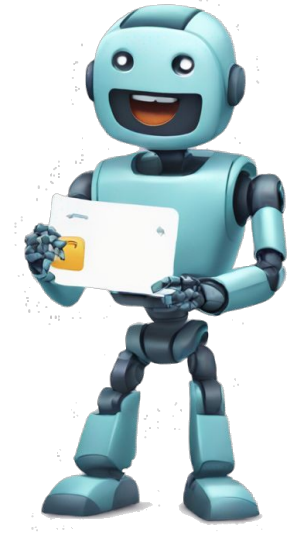
- This course is about designing **rational agents**

Definition

For each possible percept sequences, a rational agent should select an action that is expected to maximise its performance measure, given the evidence provided by the percept sequence and whatever built-in knowledge the agent has.

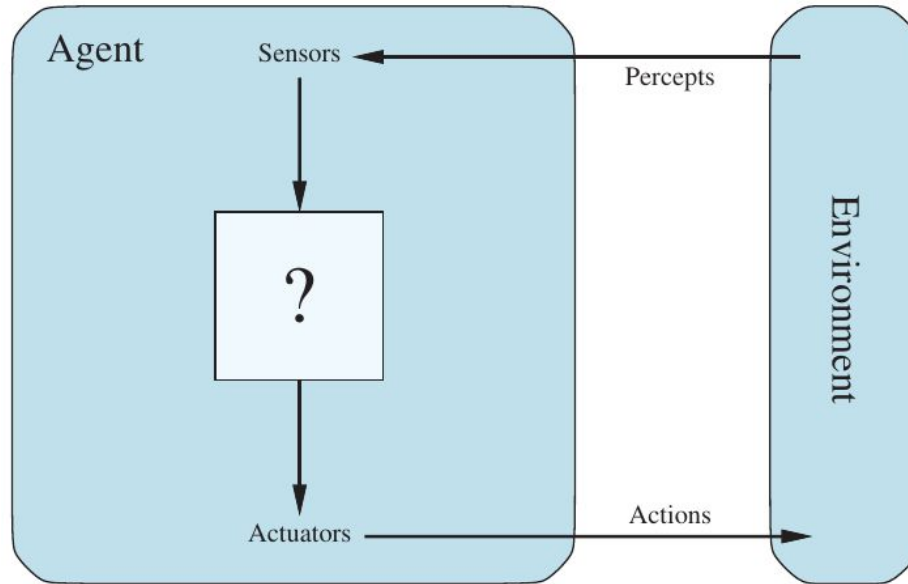
Rational agents

- **PROBLEM:** uncertainty and computational limitations make perfect rational agent(s) unachievable
- **GOAL:** design best program given the available information and machine resources!

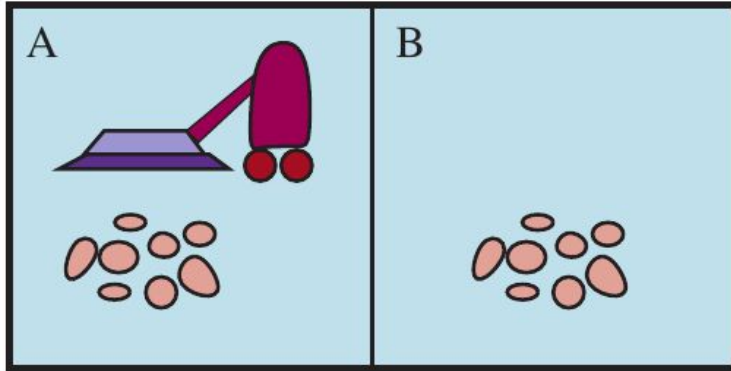


Agent and environment

- An agent is an entity that perceives and acts in its environment



Vacuum cleaner



Percept sequence	Action
[A, Clean]	Right
[A, Dirty]	Suck
[B, Clean]	Left
[B, Dirty]	Suck
[A, Clean], [A, Clean]	Right
[A, Clean], [A, Dirty]	Suck
⋮	⋮
[A, Clean], [A, Clean], [A, Clean]	Right
[A, Clean], [A, Clean], [A, Dirty]	Suck
⋮	⋮

Vacuum cleaner

- In the previous example the agent's action depends only on the last percept
 - location *Dirty* → *Suck*
 - location *Clean* → *Move*

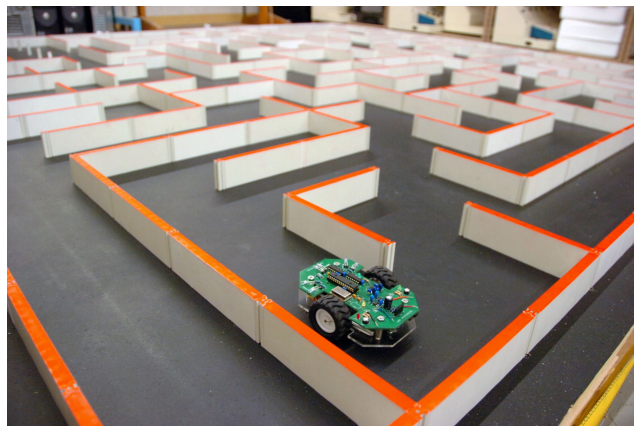
but this is not the only possibility...

- E.g., the location might remain dirty even after the sucking, and the agent could try a couple of times before moving on, independently from the results
 1. location *Dirty* → *Suck*
 2. location *Dirty* → *Suck*
 3. location (still) *Dirty* or *Clean* (whatever!) → *Move*

PEAS

- For any given class of environments and tasks, we seek the agent (or class of agents) with the best performance
- The agent's rationality depends on
 - Metric to measure **Performance**
 - Prior knowledge of the **Environment**
 - Possible agent's **Actions**
 - **Sensors** to perceive the environment

⇒ **PEAS**



PEAS example: self-driving taxi



- Metric to measure **Performance**
 - Getting to the correct destination
 - Minimizing costs (fuel consumption, tolls, car's wear and tear)
 - Maximise profits, but also safety and comfort
- Prior knowledge of the **Environment**
 - Rural lanes, urban alleys, or multi-lane highway?
 - Traffic rules, pedestrians, road works, etc.
 - Passengers' destinations
- Possible agent's **Actions**
 - Accelerator, steering wheel, brakes
 - Display or voice synthesizer to talk to passengers
 - Some way to communicate with other vehicles
- **Sensors** to perceive the environment
 - GPS, video cameras, range sensors
 - Speedometer, accelerometer
 - Engine, fuel, and electrical system sensors
 - Touchscreen or voice input for passengers

Task environment

- **Task environment:**
problem to which the rational agent tries to provide a solution
 - Each task environment defined by PEAS
- Many possible examples →

Agent Type	Performance Measure	Environment	Actuators	Sensors
Medical diagnosis system	Healthy patient, reduced costs	Patient, hospital, staff	Display of questions, tests, diagnoses, treatments	Touchscreen/voice entry of symptoms and findings
Satellite image analysis system	Correct categorization of objects, terrain	Orbiting satellite, downlink, weather	Display of scene categorization	High-resolution digital camera
Part-picking robot	Percentage of parts in correct bins	Conveyor belt with parts; bins	Jointed arm and hand	Camera, tactile and joint angle sensors
Refinery controller	Purity, yield, safety	Refinery, raw materials, operators	Valves, pumps, heaters, stirrers, displays	Temperature, pressure, flow, chemical sensors
Interactive English tutor	Student's score on test	Set of students, testing agency	Display of exercises, feedback, speech	Keyboard entry, voice

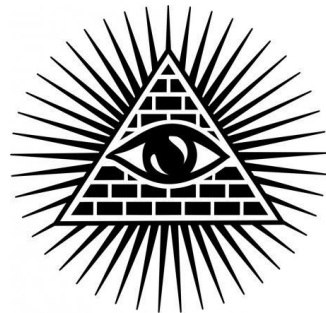
Properties of task environment

- Different properties
 - **Fully** or **Partially Observable**
 - **Deterministic** or **Stochastic**
 - **Episodic** or **Sequential**
 - **Static** or **Dynamic**
 - **Discrete** or **Continuous**
 - **Single-** or **Multi-Agent**
 - **Known** or **Unknown**

Fully vs partially observable

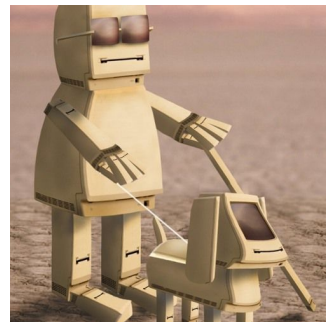
FULLY OBSERVABLE

- Agent's sensors give access to **complete state** of environment
 - sensors detect all aspects that are **relevant** to the choice of action
- No need to maintain any internal state to keep track of the world



PARTIALLY OBSERVABLE

- Typically because of **missing, noisy, or inaccurate** sensors data
 - E.g. a vacuum with broken dirt sensor
- No sensors at all $\Rightarrow$ environment is **unobservable**
 - But agent's goals may still be achievable



Deterministic vs stochastic

DETERMINISTIC

- Environment's state completely determined by current state + agent's actions
 - E.g. vacuum world is deterministic (but variations could make it not so...)



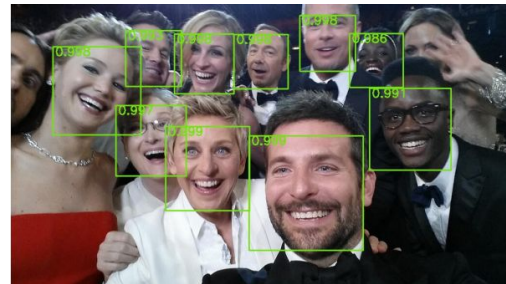
NON-DETERMINISTIC (or STOCHASTIC)

- Environment's state not (completely) predictable
- Deterministic environment that is partially observable → appears as non-deterministic
- Most real situations are treated as non-deterministic
 - E.g. Taxi driving is non-deterministic – cannot predict behavior of traffic exactly
- **Stochastic** – particular case of non-deterministic handled with **probability**

Episodic vs sequential

EPISODIC

- Agent's experience divided into atomic episodes with single percept and action
- Episode at time T does not depend on previous actions taken at $t < T$
 - E.g. **classification** tasks, like face detection on a camera shot



SEQUENTIAL

- **Current decision could affect future ones**
 - E.g. taxi driver taking a highway instead of a country road
 - More complicated than episodic because agent needs to think about consequences
- 

Static vs dynamic

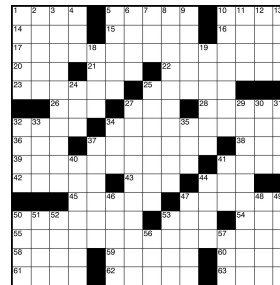
DYNAMIC

- Environment can change during agent's deliberation
 - E.g. taxi driving is dynamic – both taxi and other cars keep moving



STATIC

- Environment does not change
- Easy to deal with – no need to check world while deciding
 - E.g. crossword puzzles are static

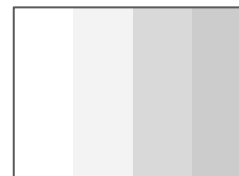


- If environment does not change but agent's performance score does ⇒ **semi-dynamic** environment

Discrete vs continuous

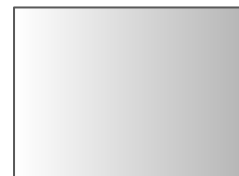
DISCRETE

- States, percepts, and actions change at **discrete time** intervals
- Typically a finite number of states / percepts / actions
 - E.g. previous vacuum cleaning world



CONTINUOUS

- One or more aspects changing continuously in time
 - E.g. taxi driver smoothly accelerating and steering the wheel
- Some input might be discrete (e.g. camera frames) but treated as continuous



Single vs multi-agent

SINGLE AGENT

- Only one relevant agent and its environment
 - E.g. agent solving a crossword puzzle
- If more than one, agent's goal is in any case not affected by other agents' goals



MULTIPLE-AGENT

- Two or more agents where A treats B as an agent
 - also B tries to maximize a performance measure, whose value depends on A's behavior
- Distinguish between (or mix of)
 - **Competitive** multiagent environment (e.g. both A and B tries to win the race...)
 - **Cooperative** multiagent environment (... but avoiding collisions)



Known vs unknown

(refers to agent's knowledge, rather than actual environment)

KNOWN

- Outcomes (or their probabilities) for all actions are given
- Environment could be known but partially observable
 - E.g. solitaire card game – known rules but unknown hidden cards



UNKNOWN

- Outcomes of actions are not known a priori
- Agent will have to learn them
- Unknown environment could nevertheless be fully observable
 - E.g. video game (completely observable) but unknown use of joystick buttons



Example: computer chess



Observable

Partially observable

Fully observable



Observable

Partially observable



Fully observable





Deterministic

Deterministic



Stochastic



Episodic

Episodic

Sequential



Episodic

Episodic

0%

Sequential

0%



Static

Static

Dynamic



Static

Static

0%

Dynamic

0%



Discrete

Continuous



Discrete



Continuous



Single/multi-agent

Single-agent

Multi-agent



Single/multi-agent

Single-agent



Multi-agent



Known

Known

Unknown



Known

Known



0%

Unknown



0%



Examples: computer chess



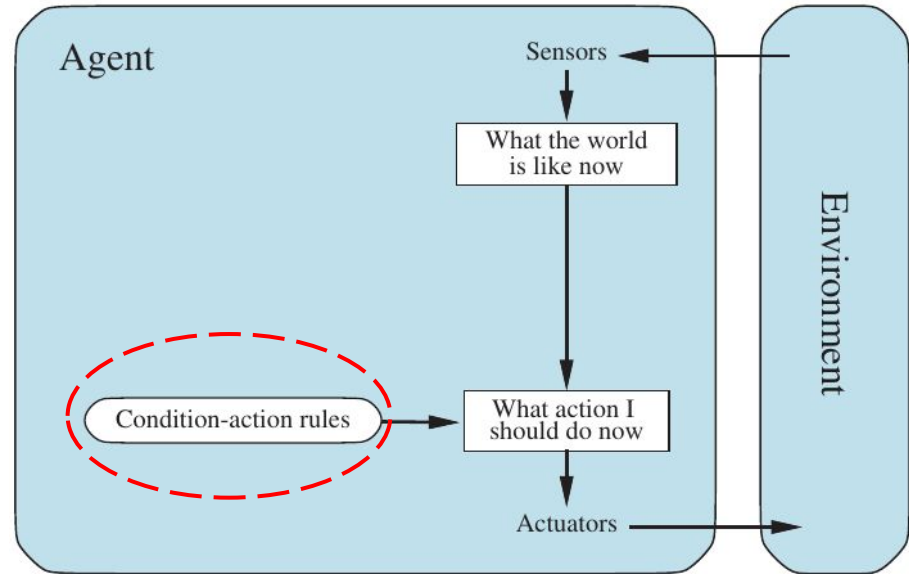
- **Properties**
 - **Fully observable**
 - **Deterministic**
 - **Sequential**
 - **Static (semi-dynamic with clock)**
 - **Discrete**
 - **Multi-Agent**
 - **Known**

Types of agents

- Simple reflex agents
- Model-based reflex agents
- Goal-based agents
- Utility-based agents
- Learning agent

Reflex agent

- Agent's behaviour based on simple **condition-action (if-then) rules**
- Decisions depends only on current percepts:
 - how does the world look right now?
⇒ needs full observability!
- History of percepts is ignored
⇒ small program!



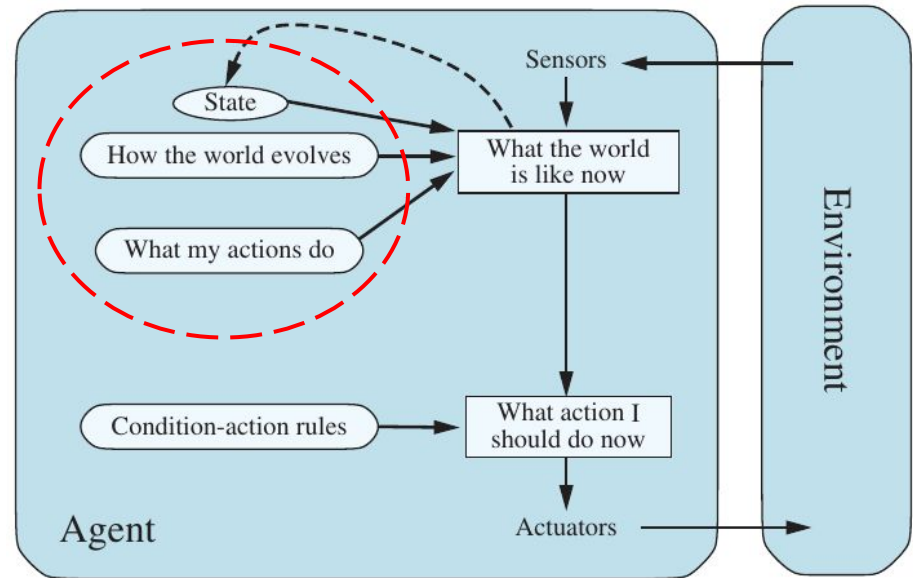
Reflex agent: example

- Early versions of robot cleaners (e.g. Roomba) implemented a very basic behaviour – move until you bump against something, then turn and restart



Model-based reflex agent

- Compared to the previous one, this maintains an **internal state** (e.g. previous position)
 - good for partial observability
- It needs a **transition model** to know how the world changes in response to an agent's action...
- ... plus a **sensor model** to link actual percept to current state

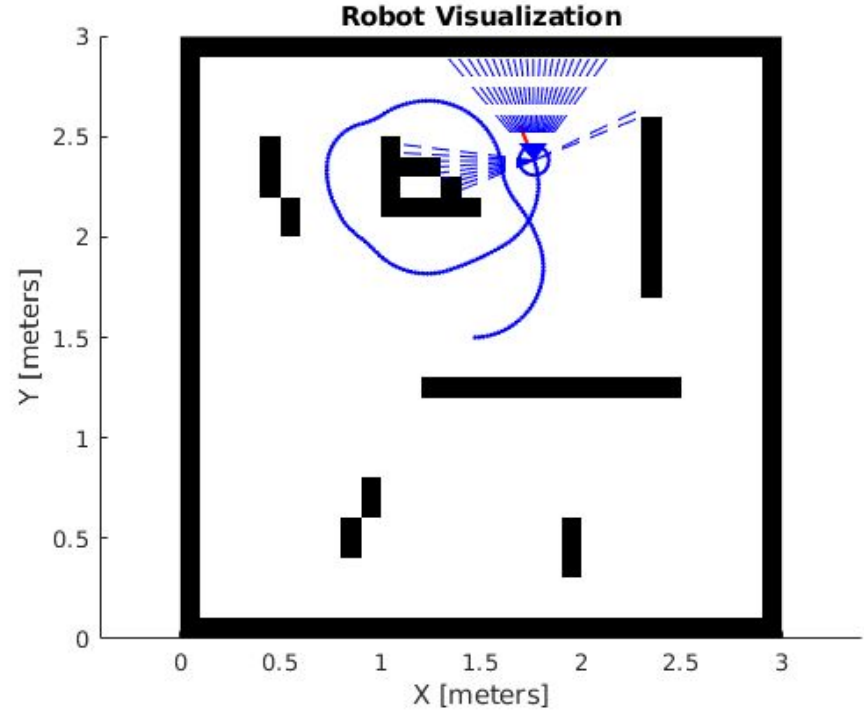


Model-based reflex agent: example

- Robot estimates its position (state) so it can move to a different one

E.g. **Kalman filter-based localization**

- 1) Using a transition (motion) model, the robot guesses its current position based on the previous one
- 2) With a sensor model, it guesses also the current percepts and compare them to the actual ones
- 3) The difference between guessed and actual percepts at step 2) is used to correct the state at step 1)

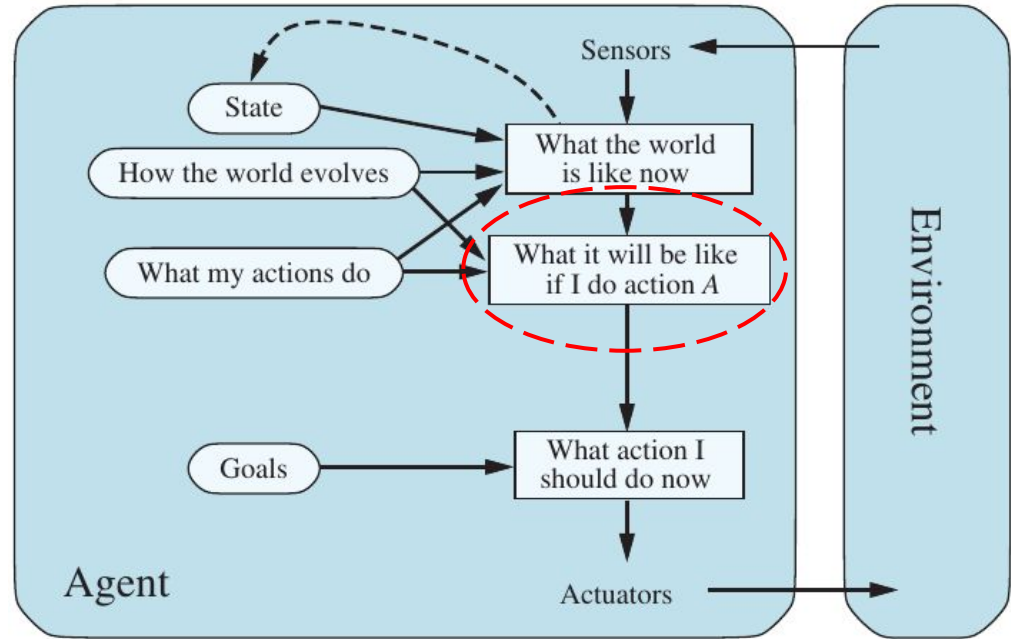


Goal-based agent

- In many cases knowing the world is not enough
- Actions not just for immediate effects (like reflex), but also for long-term goals

⇒ need of algorithms for **search and planning**

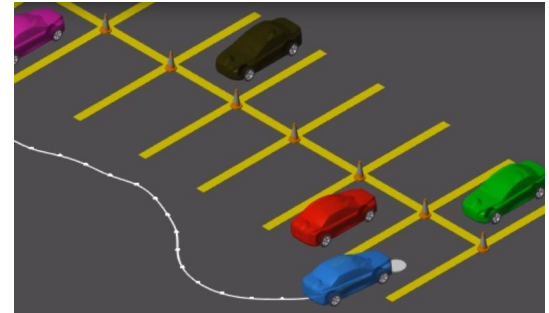
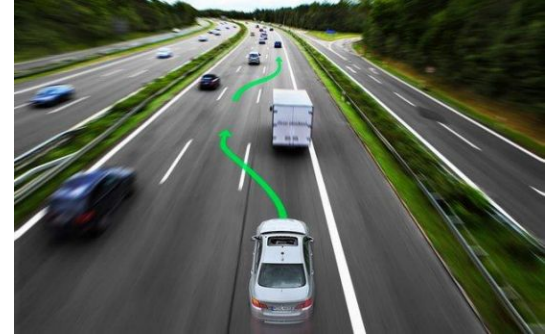
- Goals can be changed
⇒ more flexibility



Goal-based agent: example

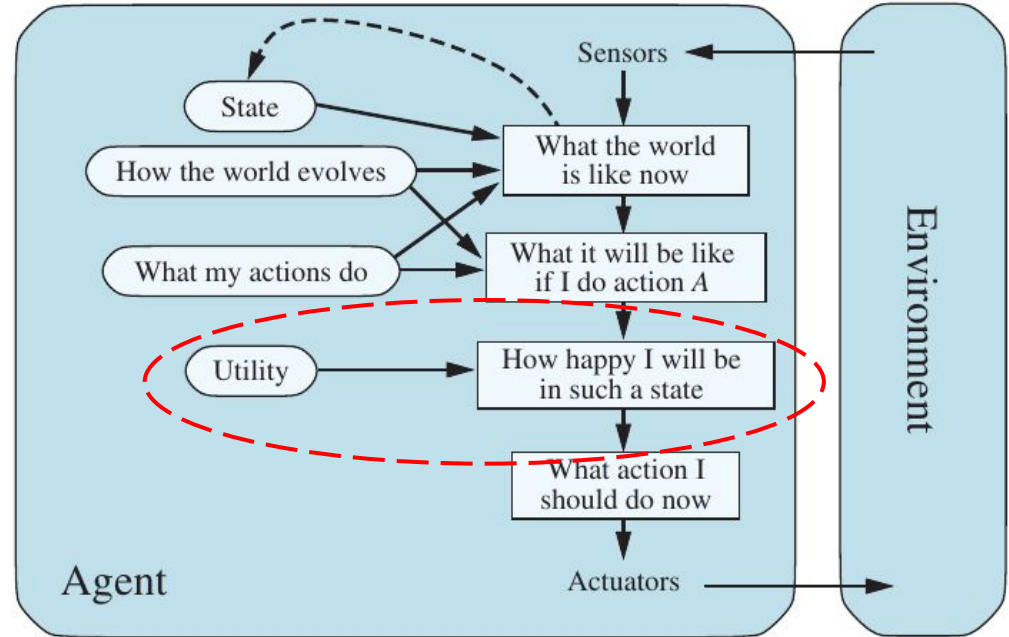
E.g. autonomous car

- Self-driving agent that plans best sequence of actions to ***overtake*** another vehicle (goal)
- Uses own state and knowledge of environment to search among possible solutions towards the goal
- Similar approach (but different goal) could be used for ***parking***



Utility-based agents

- A goal do not specify how good a sequence of actions or the goal itself are
- A performance measure should be used to score the states of the agent
- Using an **utility function**, the agent's can be directed towards states of hypothetical **maximum performance**



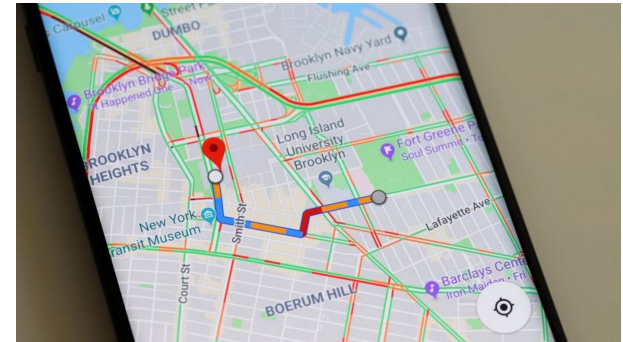
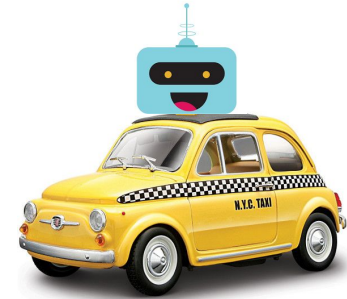
Utility-based agents

- Useful when
 - there are conflicting goals (or paths to a goal)
 - not sure whether goals can be achieved (uncertainty)
- Rational agent maximises the **expected utility**
- E.g. a utility-based agent could choose whether to draw or not any card in a poker game depending on the expected outcome



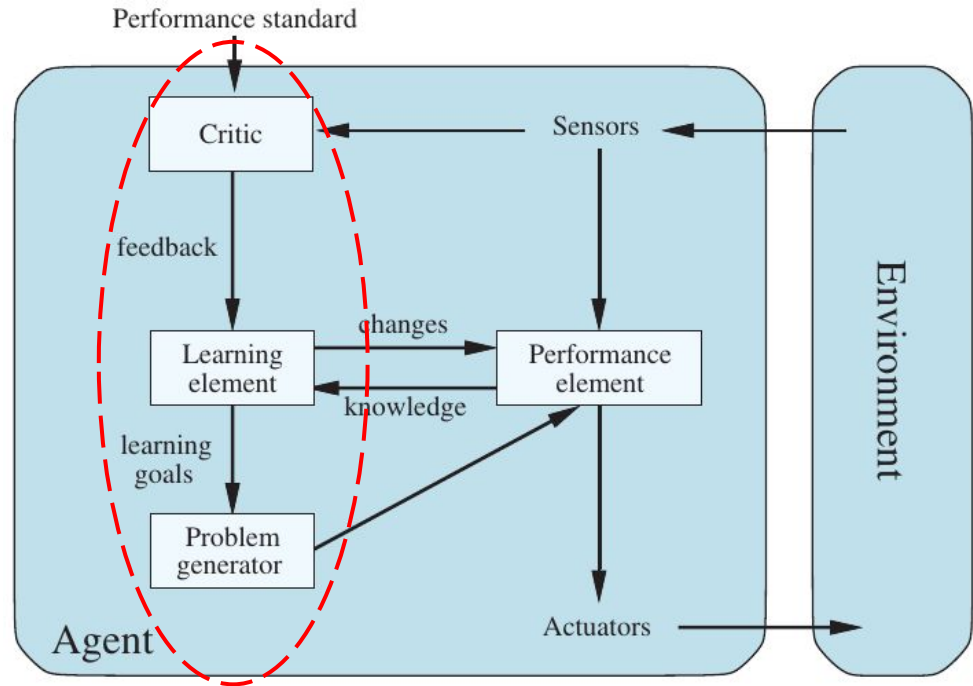
Example: self-driving taxi

- **Reflex** agent: taxi brakes whenever front car's lights are red (but then it could always be braking...)
- **Model-based reflex** agent: taxi brakes only when it's moving and there is a transition from low- to high-intensity of the front car's lights, if it cannot change lane, etc.
- **Goal-based** agent: taxi drives towards a particular destination...
- **Utility-based** agent: ... taking into account fuel consumption (i.e. shortest path and time)



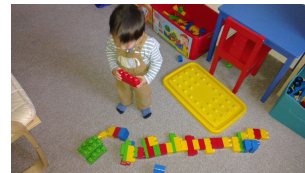
Learning agent

- Agent learns how to become more competent through experience
- Can deal with initially unknown environments or dynamic changes
- Four main elements:
 - **Performance** – to select actions (basically any of the previous agents)
 - **Learning** – to make improvements
 - **Critic** – to evaluate outcome
 - **Problem generator** – to suggest new “exploratory” actions



Learning agent: examples

- Humans learn from (supervised) teaching and autonomous exploration
- A robot football-player could learn from free-kick success rate
- A robotaxi could learn to improve based on the customers' tips
- Recommender systems can adapt to customers (through reinforcement learning)



Summary

- An agent is something (a program) perceiving and acting in an environment
- Its behaviour depends on PEAS (performance, environment, actions and sequence of percepts)
- Task environment can have different properties (observable, stochastic, etc.)
- There are different agent architectures of increasing complexity

Conclusions

- Suggested reading

Chapter 2, sec. 2.1-2.4 of Russell & Norvig

- Next lecture

Solving problems by searching

- Questions?

